

ModelScope 开源大语言模型部署与横向对比实验报告

《人工智能导论》第三次作业

姓名： 蒋昊沅

学号： 2450333

课程： 人工智能导论

项目链接： github.com/JambitX11/LLM-Deployment-and-Comparison

目录

1. 实验目的	3
2. 实验平台与环境	3
3. 模型选择	5
4. 实验方法	6
5. 仓库结构	6
6. 问答测试结果	7
6.1. Qwen-7B-Chat	7
6.2. ChatGLM3-6B	8
6.3. Baichuan2-7B-Chat	8
7. 横向对比分析	9
8. 实验中遇到的问题与解决方法	10
9. 总结	11

1. 实验目的

本次实验围绕 ModelScope/魔搭平台上的开源大语言模型部署与测试展开。实验目标是在云端 Notebook 环境中完成模型仓库下载、Python 依赖安装和命令行推理，并使用同一组中文测试问题比较不同模型在中文表达、语义歧义理解、多层逻辑推理、指代关系理解以及一词多义理解方面的表现。通过横向对比，可以更直观地观察不同开源大语言模型在相同任务下的能力差异和运行代价。

本实验测试了 Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat 三个模型。测试问题并不只考察回答是否通顺，更关注模型能否理解中文中的特殊语境。例如，“冬天能穿多少穿多少”和“夏天能穿多少穿多少”字面一致，但实际语境含义相反；“谁都看不上”也可能因为主客体不同而产生幽默效果。这类问题能够较集中地暴露模型在中文语义理解上的差异。

2. 实验平台与环境

实验在 ModelScope/魔搭平台的 CPU Notebook 环境中完成。项目代码通过 GitHub 克隆到 `/mnt/workspace/LLM-Deployment-and-Comparison`，模型文件下载到 `/mnt/data/`。由于云平台磁盘空间有限，三个模型没有同时保存在 `/mnt/data/` 中，而是采用“下载一个模型、测试一个模型、保存结果和截图、必要时删除后再下载下一个模型”的方式完成实验。

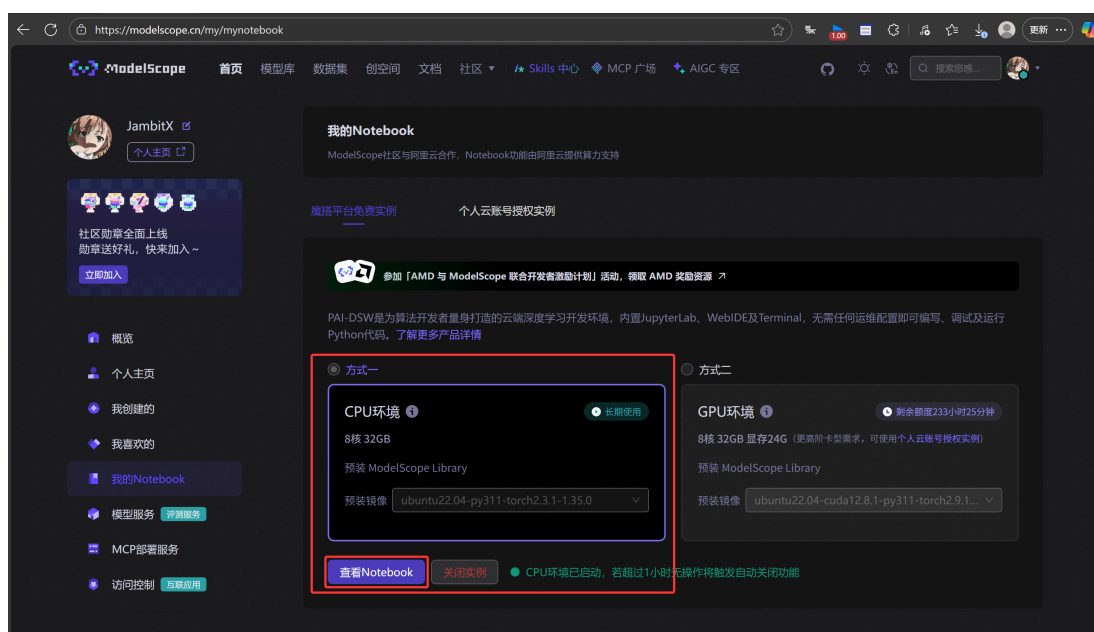


图 1 ModelScope Notebook 页面

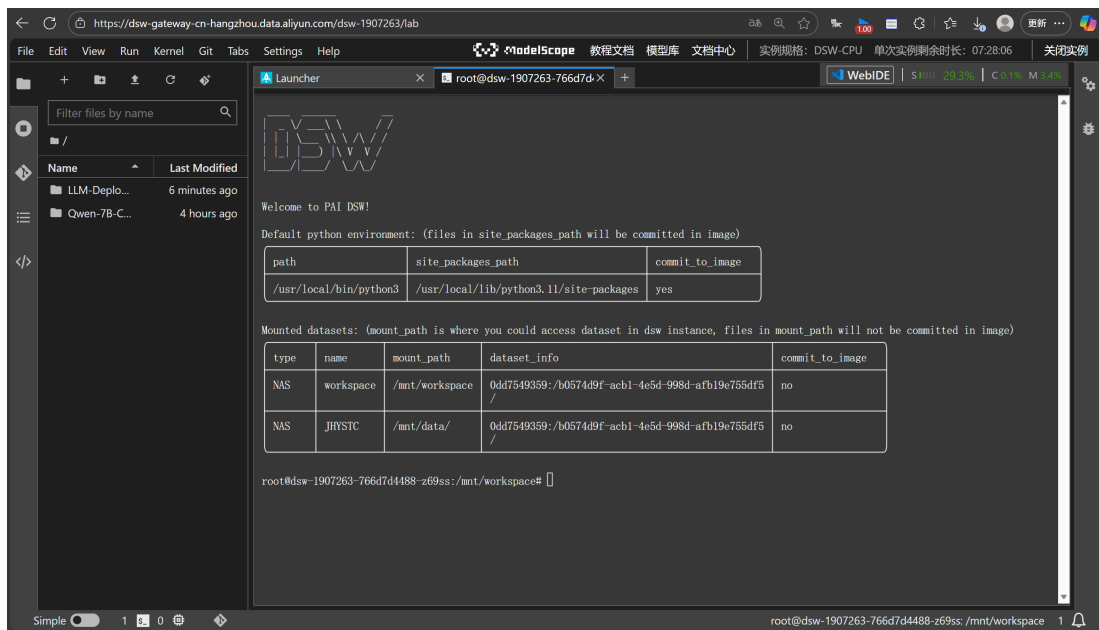


图 2 Notebook Terminal 页面

项目仓库克隆过程如下图所示。克隆完成后，后续安装依赖、下载模型和运行测试脚本都在该项目目录下完成。



图 3 克隆 GitHub 项目

依赖安装通过仓库中的 `scripts/install_deps_cpu.sh` 完成。该脚本安装 CPU 版本 PyTorch，并安装 `transformers`、`modelscope`、`sentencepiece`、`accelerate` 等推理所需依赖。实验过程中曾遇到 Qwen 相关依赖版本兼容问题，因此最终在脚本中固定了 `transformers==4.33.3` 和 `transformers_stream_generator==0.0.4`。

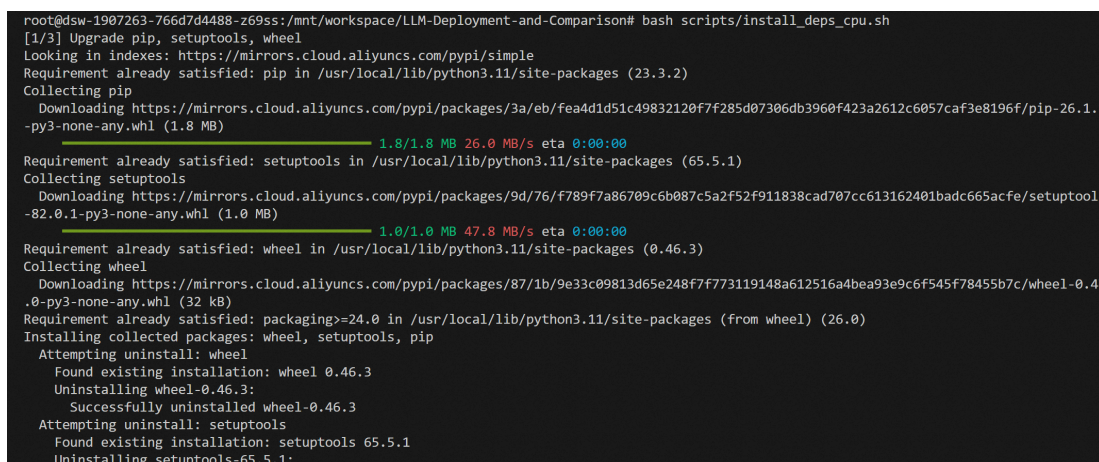


图 4 依赖安装过程

```
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# pip install torch==2.3.0+cpu torchvision==0.18.0+cpu --index-url https://download.pytorch.org/whl/cpu
Looking in indexes: https://download.pytorch.org/whl/cpu
Collecting torch==2.3.0+cpu
  Downloading torch-2.3.0%2Bcpu-cp311-cp311-linux_x86_64.whl (190.4 MB)
    190.4/190.4 MB 1.2 MB/s 0:02:37
Collecting torchvision==0.18.0+cpu
  Downloading torchvision-0.18.0%2Bcpu-cp311-cp311-linux_x86_64.whl (1.6 MB)
    1.6/1.6 MB 1.2 MB/s 0:00:01
Requirement already satisfied: filelock in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.25.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (4.15.0)
Requirement already satisfied: sympy in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (1.14.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.11/site-packages (from torch==2.3.0+cpu) (2025.3.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/site-packages (from torchvision==0.18.0+cpu) (1.26.4)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.11/site-packages (from torchvision==0.18.0+cpu) (11.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/site-packages (from jinja2->torch==2.3.0+cpu) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/site-packages (from sympy->torch==2.3.0+cpu) (1.3.0)
Installing collected packages: torch, torchvision
Attempting uninstall: torch
  Found existing installation: torch 2.9.1+cpu
  Uninstalling torch-2.9.1+cpu:
    Successfully uninstalled torch-2.9.1+cpu
Attempting uninstall: torchvision
  Found existing installation: torchvision 0.24.1+cpu
  Uninstalling torchvision-0.24.1+cpu:
    Successfully uninstalled torchvision-0.24.1+cpu
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
torch-einops-utils 0.0.30 requires torch>=2.5, but you have torch 2.3.0+cpu which is incompatible.
torchaudio 2.9.1+cpu requires torch==2.9.1, but you have torch 2.3.0+cpu which is incompatible.
hyper-connections 0.4.9 requires torch>=2.5, but you have torch 2.3.0+cpu which is incompatible.
Successfully installed torch-2.3.0+cpu torchvision-0.18.0+cpu
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
```

图 5 依赖安装完成

3. 模型选择

本实验选择了三个中文开源大语言模型：Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat。它们都属于中文场景中较常见的开源对话模型，参数规模接近，适合在同一组测试问题上进行横向比较。

模型	本地模型目录	推理脚本	说明
Qwen-7B-Chat	/mnt/data/Qwen-7B-Chat	scripts/run_qwen_cpu.py	中文能力较强，支持 chat / chat_stream 接口。
ChatGLM3-6B	/mnt/data/chatglm3-6b	scripts/run_chatglm3_cpu.py	典型中文对话模型，模型加载较快，但本次 CPU 生成耗时较长。
Baichuan2-7B-Chat	/mnt/data/Baichuan2-7B-Chat	scripts/run_baichuan_cpu.py	中文开源对话模型，输出速度较快，但部分问题存在偏题。

模型下载截图如下。

```
root@dsw-1906671-74d5885f76-j6x41:/mnt# cd data
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# git clone https://www.modelscope.cn/qwen/Qwen-7B-Chat.git
正克隆到 'Qwen-7B-Chat'...
remote: Enumerating objects: 554, done.
remote: Counting objects: 100% (56/56), done.
remote: Compressing objects: 100% (30/30), done.
remote: Total 554 (delta 30), reused 49 (delta 26), pack-reused 498
接收对象中: 100% (554/554), 16.47 MiB | 27.11 MiB/s, 完成.
处理 delta 中: 100% (294/294), 完成.
过滤内容: 100% (8/8), 14.38 GiB | 154.33 MiB/s, 完成.
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# ls
Qwen-7B-Chat
```

图 6 Qwen 模型下载

```
root@dsw-1906671-74d5885f76-j6x41:/mnt/data# git clone https://www.modelscope.cn/ZhipuAI/chatglm3-6b.git
正克隆到 'chatglm3-6b'...
remote: Enumerating objects: 147, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 147 (delta 0), reused 0 (delta 0), pack-reused 140
接收对象中: 100% (147/147), 77.69 KiB | 544.00 KiB/s, 完成.
处理 delta 中: 100% (64/64), 完成.
过滤内容: 100% (15/15), 23.26 GiB | 157.70 MiB/s, 完成.
```

图 7 ChatGLM3 模型下载

```

root@dsw-1906671-74d5885f76-j6x4l:/mnt/data# git clone https://www.modelscope.cn/baichuan-inc/Baichuan2-7B-Chat.git
正克隆到 'Baichuan2-7B-Chat'...
remote: Enumerating objects: 121, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 121 (delta 6), reused 0 (delta 0), pack-reused 100
接收对象中: 100% (121/121), 472.62 KiB | 4.18 MiB/s, 完成.
处理 delta 中: 100% (51/51), 完成.

```

图 8 Baichuan 模型下载

4. 实验方法

测试问题保存在 `prompts/test_questions.json` 中，人工阅读版说明保存在 `prompts/test_questions.md` 中。为了避免 CPU 环境下重复加载模型造成额外耗时，实验使用 `scripts/run_questions_cpu.py` 对单个模型进行批量测试。脚本每次只加载一个模型，然后按顺序运行 5 个测试问题，并将输出写入 `results/raw_outputs/` 下的 Markdown 文件。

三个模型的主要运行命令如下：

```

python scripts/run_questions_cpu.py --model qwen --max_new_tokens 256 --dtype auto --
num_threads 4
python scripts/run_questions_cpu.py --model chatglm3 --max_new_tokens 256 --dtype auto --
num_threads 4
python scripts/run_questions_cpu.py --model baichuan --max_new_tokens 256 --dtype auto --
num_threads 4

```

实验问题覆盖五类中文能力。第一题考察模型能否理解同一句话在冬天和夏天的相反语境；第二题考察“谁都看不上”的双向歧义；第三题考察“知道/不知道”的多层逻辑；第四题考察“明明、白白、他、她”的断句和指代关系；第五题考察“意思”一词在不同语境中的多义解释。

5. 仓库结构

本仓库按“脚本、测试问题、结果、截图、报告、文档”组织。代码和实验材料都保存在 GitHub 中，模型权重不进入仓库，而是在云平台运行时下载到 `/mnt/data/`。这样的组织方式既能满足作业提交中的公开仓库要求，也避免将大模型权重上传到 GitHub。

```

LLM-Deployment-and-Comparison/
├── README.md
├── requirements.txt
├── scripts/
│   ├── install_deps_cpu.sh
│   ├── download_models.sh
│   ├── run_questions_cpu.py
│   ├── run_qwen_cpu.py
│   ├── run_chatglm3_cpu.py
│   └── run_baichuan_cpu.py
├── prompts/
│   ├── test_questions.md
│   └── test_questions.json
├── results/
│   ├── raw_outputs/
│   ├── qwen_results.md
│   ├── chatglm3_results.md
│   ├── baichuan_results.md
│   └── comparison_table.md
├── screenshots/
├── docs/
└── report/
    ├── 实验报告.md
    ├── 实验报告.typ
    └── 实验报告.pdf

```


scripts/ 目录保存依赖安装、模型下载和模型推理脚本。其中 run_questions_cpu.py 是主要批量测试脚本，会读取 prompts/test_questions.json 中的五个问题，并将输出写入 results/raw_outputs/。results/ 目录保存整理后的单模型结果和横向对比表，screenshots/ 保存三组模型各五个问题的运行截图，report/ 保存 Markdown、Typst 和 PDF 形式的实验报告。

6. 问答测试结果

从运行耗时看，三个模型差异明显。Qwen 加载和生成都比较稳定，Baichuan 加载较慢但生成速度较快，ChatGLM3 虽然加载最快，但生成阶段耗时远高于另外两个模型。

模型	模型加载耗时	5 题生成总耗时	运行总耗时
Qwen-7B-Chat	3.37 秒	192.79 秒	196.16 秒
ChatGLM3-6B	0.86 秒	3853.97 秒	3854.83 秒
Baichuan2-7B-Chat	100.20 秒	200.43 秒	300.63 秒

6.1. Qwen-7B-Chat

Qwen-7B-Chat 的模型加载耗时为 3.37 秒，5 个问题生成总耗时为 192.79 秒，运行总耗时为 196.16 秒。从耗时看，Qwen 在本次 CPU 环境下运行相对顺利，五个问题中耗时最长的是“一词多义”问题，生成耗时为 64.92 秒。

问题	生成耗时	回答表现概述
问题 1	45.43 秒	能识别冬天应多穿，但对夏天解释为“多穿防晒和保持凉爽”，没有准确说出夏天语境中“能穿多少穿多少”常指尽量少穿。
问题 2	27.53 秒	没有准确抓住两个“谁都看不上”分别表示“自己看不上别人”和“别人看不上自己”的幽默歧义。
问题 3	35.27 秒	对多层“知道/不知道”关系解释混乱，没有给出稳定明确的逻辑结论。
问题 4	19.64 秒	没能完成断句和指代解析，回答为上下文不足。
问题 5	64.92 秒	对“意思”的多义解释较完整，能区分意图、敷衍、够朋友和小事等含义。

```
root@ds0-1509780-7467c09449-vdm2:/mnt/workspace/LIN-Deployment-and-Comparison# python scripts/run_questions_cpu.py --model qwen --max_new_t
shen 256 --num_threads 4
[INFO] Loading Qwen-7B-Chat from /mnt/data/qwen-7b-chat
[INFO] This may take several minutes on CPU.
[INFO] dtype=auto, torch_run_threads=4
/usr/local/lib/python3.11/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils._pytree._register_pytree_node is depreca
ted. Please use torch.utils._pytree._register_pytree_node instead.
  torch.utils._pytree._register_pytree_node(
Loading checkpoint shards: 100%
[INFO] Model loaded in 3.37 seconds.
[4/5] 冬天夏天能穿多少穿多少穿多少
Prompt: 请提出以下两句话区别在哪里？1. 冬天：能穿多少穿多少 2. 夏天：能穿多少穿多少
这两句话的意思基本相同，都表达了“能穿多少穿多少”的意思，但是它们的语境不同。第一句是在描述冬天的情况，强调在寒冷的天气里应该多穿衣服来保
暖；而第二句是在描述夏天的情况，强调在炎热的天气里应该多穿衣服来防晒和保持凉爽。因此，尽管这两句话的意思相近，但它们的语境和用法略有不同。
[TIME] Question 1: 45.43 seconds
```

Qwen 问题 1

```
[3/5] 谁能看不上谁
Prompt: 请解释这句话中两个“谁都看不上”的区别：单身时产生的原因有两个，一是谁都看不上，二是谁都看不上。
在这句话中，“谁都看不上”是一个成语，意思是没有人喜欢或者接受。而第二个“谁都看不上”则是在解释单身时产生的第一个原因时使用的，表示是因为没有合
适的人选才导致单身。所以，这两个“谁都看不上”的含义是不同的。
[TIME] Question 2: 27.53 seconds
```

Qwen 问题 2

```
[3/5] 明知道不知道
Prompt: 明知道不知道你知道他不知道吗？这句话里，到底谁不知道？
在这个句子中，有两个人：他知道（这个动作的发出者）、你和他知道（这两个动作的接收者）。由于这个句子没有明确指出“他知道”什么，所以我们可以认为
他可能不知道任何事。因此，可以这样理解：他知道你知道他知道他知道，也就是说，他知道他知道他知道他知道，但是需要注意的是，这仅是基于我们对
这个句子的理解，而实际情况可能会略有不同。
[TIME] Question 3: 35.27 seconds
```

Qwen 问题 3

```
[4/5] 明明和白白谁喜欢谁
Prompt: 明明明明明白白白喜欢谁，可她就是不说，明明和白白谁喜欢谁？
很抱歉，我无法确定哪个人是“明明”和“白白”，因为你没有提供足够的上下文信息。如果你能提供更多信息，我将尽力回答你的问题。
[TIME] Question 4: 19.64 seconds
```

Qwen 问题 4

```
[5/5] “意思”的不同含义
Prompt: 请解释下面对话中每个“意思”的不同含义：甲：你这是什么意思？乙：没什么意思，就是意思意思。甲：你这就不够意思了。乙：小意思，小意思。
在这个对话中，“意思”这个词有不同的含义：
1. 甲问：“你这是什么意思？”这里的“意思”是询问对方说话的真实意图或目的。
2. 乙回答：“没什么意思，就是意思意思。”这里的“意思”是一种模糊不清、含糊其辞的回答方式，可能是用来避免直接回答问题或者敷衍塞责。
3. 甲说：“你这就不够意思了。”这里的“意思”是指礼貌程度、道谢准则等方面的意思，表示乙的回答不够尊重或真诚。
4. 乙回答：“小意思，小意思。”这里的“意思”是指事情不大、不重要，或者表达一种谦虚的态度。
[TIME] Question 5: 64.92 seconds
Used time: 3.37 seconds
Generation time: 192.79 seconds
Total measured time: 196.16 seconds
Note: Results saved to results/raw_outputs/qwen_all_questions.md
```

Qwen 问题 5

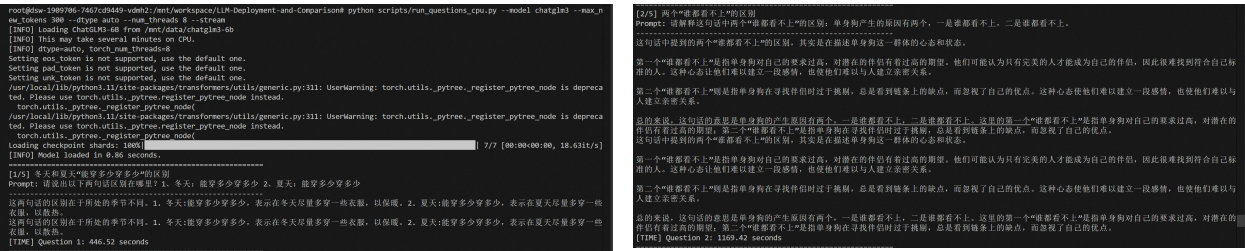
图 9 Qwen 五个问题运行截图

总体来看，Qwen 的中文表达比较自然，格式也较清楚，但在本组偏歧义、偏脑筋急转弯的问题上不够稳定。它对第五题的解释最好，能够较系统地列出“意思”的不同含义；但在第二题和第四题上没有抓住题目的关键语言现象，说明其对中文双关和非常规断句的鲁棒性仍有限。

6.2. ChatGLM3-6B

ChatGLM3-6B 的模型加载耗时为 0.86 秒，5 个问题生成总耗时为 3853.97 秒，运行总耗时为 3854.83 秒。它加载很快，但生成极慢，尤其是第二题和第五题都超过 1000 秒。该现象说明在 CPU 环境下，模型加载速度和文本生成速度并不一定一致，生成阶段才是主要耗时来源。

问题	生成耗时	回答表现概述
问题 1	446.52 秒	能指出冬天多穿保暖，但夏天仍解释为“多穿散热”，没有准确理解夏天应尽量少穿。
问题 2	1169.42 秒	回答很长，但没有抓住“自己看不上别人/别人看不上自己”的核心双关，解释偏向“要求过高”和“挑剔”。
问题 3	805.07 秒	将问题解释成悖论，逻辑链条不够准确，没有直接回答谁不知道。
问题 4	397.53 秒	识别到句子模糊，但对“明明”和“白白”的关系判断摇摆，结论不稳定。
问题 5	1035.43 秒	能尝试解释不同“意思”，但部分角色和语义分配不够准确。



ChatGLM3 问题 1

ChatGLM3 问题 2

ChatGLM3 问题 4

ChatGLM3 问题 3

ChatGLM3 问题 5

图 10 ChatGLM3 五个问题运行截图

ChatGLM3 的回答风格偏解释型，常常会展开较长分析，但长回答没有必然带来更高准确率。第二题和第三题中，它给出了较多文字，却没有抓住题目真正考察的语言歧义和逻辑关系。在本次 CPU 环境中，ChatGLM3 的生成耗时远高于另外两个模型，因此从作业展示和实验效率角度看，它的运行成本较高。

6.3. Baichuan2-7B-Chat

Baichuan2-7B-Chat 的模型加载耗时为 100.20 秒，5 个问题生成总耗时为 200.43 秒，运行总耗时为 300.63 秒。它的加载时间明显长于 Qwen 和 ChatGLM3，但生成速度较快，整体耗时接近 Qwen，远低于 ChatGLM3。

问题	生成耗时	回答表现概述
问题 1	23.41 秒	正确说明冬天多穿保暖、夏天少穿散热，是三者中对该题理解最准确的回答之一。
问题 2	88.46 秒	能区分两个“谁都看不上”有不同侧重点，但仍没有明确说出“自己看不上别人”和“别人看不上自己”这一标准解释。
问题 3	11.78 秒	直接回答“他不知道”，虽然解释较简略，但结论较接近题目核心。
问题 4	18.34 秒	将关系判断为“明明喜欢白白”，与更合理的断句解释不一致。
问题 5	58.44 秒	出现扩写和偏题，额外加入原问题没有出现的对话，导致解释准确性下降。

```
root@idm-1989786-7467c29449-vsm02:/mnt/workspaces/LiH-Deployment-and-Comparison python scripts/run_questions_cpu.py --model baichuan --max_n
ew_tokens 156 --dtype auto --num_threads 4
[INFO] Loading Baichuan2-7B from /mnt/data/baichuan2-7B-Chat
[INFO] This may take several minutes on CPU.
[INFO] device=auto, torch_num_threads=4
/usr/local/lib/python3.11/site-packages/transformers/utils/generic.py:311: UserWarning: torch.utils._pytree._register_pytree_node is deprecate
d. Please use torch.utils._pytree.register_pytree_node instead.
  torch.utils._pytree._register_pytree_node(
xformers is not installed correctly. If you want to use memory_efficient_attention to accelerate training use the following command to insta
ll xformers.
[INFO] Model loaded in 100.28 seconds.
=====
[1/5] 冬天和夏天能穿多少穿多少的区别
Prompt: 请说出以下两句话区别在哪里？1. 冬天：能穿多少穿多少 2. 夏天：能穿多少穿多少
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:362: UserWarning: 'do_sample' is set to 'False'. How
ever, 'temperature' is set to 0.3 -- this flag is only used in sample-based generation modes. You should set 'do_sample=True' or unset 'tem
perature'.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:367: UserWarning: 'do_sample' is set to 'False'. How
ever, 'top_p' is set to 0.85 -- this flag is only used in sample-based generation modes. You should set 'do_sample=True' or unset 'top_p'.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/transformers/generation/configuration_utils.py:377: UserWarning: 'do_sample' is set to 'False'. How
ever, 'top_k' is set to 5 -- this flag is only used in sample-based generation modes. You should set 'do_sample=True' or unset 'top_k'.
  warnings.warn(
/usr/local/lib/python3.11/site-packages/torch/backends/cuda/_init_.py:342: FutureWarning: torch.backends.cuda.sdp_kernel() is deprecated.
In the future, this context manager will be removed. Please see, torch.nn.attention.sdp_kernel() for the new context manager, with updated
signature.
  warnings.warn(
这两句话的区别在于，第一句是在冬天时提出的建议，而第二句是在夏天时提出的建议。在冬天，人们通常穿更多的衣物来保暖，而在夏天，人们则穿较少的衣
物来散热。因此，这两句话的侧重点和目的有所不同。
[TIME] Question 1: 23.41 seconds
```

Baichuan 问题 1

```
[3/5] 到底谁不知道
Prompt: 他知道自己知道他知道他不知道吗？这句话里，到底谁不知道？
-----
在这个句子中，“他”不知道。因为句子的主题是“他”，而“他”不知道是句子的主要信息。
[TIME] Question 3: 11.78 seconds
```

Baichuan 问题 3

```
[5/5] "意思"的不同含义
Prompt: 请解释下面对话中每个“意思”的不同含义：甲：你这是什么意思？乙：没什么意思，就是意思意思。甲：你这就不够意思了。乙：小意思，小意思。
甲：你真是太意思了。乙：谢谢你的意思。
甲：你这是什么意思？
乙：没什么意思，就是意思意思。
这里的“意思”表示“礼貌的表示，类似于“感谢”或“客气”了”。
2. 甲：你这就不够意思了。
乙：小意思，小意思。
这里的“意思”表示谦虚，表示自己的行为还不够标准，还需要努力。
3. 甲：你真是太意思了。
乙：谢谢你的意思。
这里的“意思”表示感谢，表示对方对自己的帮助或帮助表示感谢。
[TIME] Question 5: 58.44 seconds
Load time: 100.28 seconds
Generation time: 200.43 seconds
Total measured time: 300.63 seconds
Done. Results saved to results/raw_outputs/baichuan_all_questions.md
```

Baichuan 问题 5

```
[2/5] 两个“谁都看不上”的对比
Prompt: 请解释这句话中两个“谁都看不上”的区别，单身狗产生的原因有两个，一是谁都看不上，二是谁都看不上。
-----
这句话中的两个“谁都看不上”似乎在表达相同的意思，即一个人对他人的拒绝和看不起。然而，这两个“谁都看不上”在语境和含义上略有不同。
第一个“谁都看不上”是指这个人主观意愿，表示这个人对自己的伴侣或恋爱对象有很高的要求，认为没有人能够满足他们的期望，因此不愿意接受任何人的追求。
这种心态可能是与他们自身性格上的孤独和挑剔，最终成为单身狗。
第二个“谁都看不上”则是指这个人所处的社会环境中的排他性普遍存在而导致的排他性，导致他们对所有人都感到不满。从而成为单身狗。这种原因可能源于对现实的不满，对理想爱情的期待过高或受到其他因素的限制（如家庭背景、经济条件等）。
总之，这两个“谁都看不上”在表达上略有不同，一个更侧重于主观意愿，另一个则反映社会现实和压力。两者都可能导致单身狗的产生，但原因和表现方式有所不同。
[TIME] Question 2: 88.46 seconds
```

Baichuan 问题 2

```
[4/5] 明明和白白谁喜欢谁
Prompt: 明明明明白白白白喜欢谁，可她就是不，明明和白白谁喜欢谁？
-----
从问题描述来看，明明喜欢白白，但问题是，为什么明明不喜欢白白却不说出来呢？这可能是明明害怕白白拒绝，或者是不想破坏他们之间的友谊。当然，这只是一个猜测，实际情况可能因人而异。
[TIME] Question 4: 18.34 seconds
```

Baichuan 问题 4

图 11 Baichuan 五个问题运行截图

Baichuan 的优势在于回答速度较快，并且第一题和第三题的结论比较直接。但它在复杂语境中容易自行补充不存在的内容，第五题就是典型例子。对话中原本只有四句，它却额外生成“你真是太意思了”“谢谢你的意思”等内容，导致分析对象偏离原题。

7. 横向对比分析

对比维度	Qwen-7B-Chat	ChatGLM3-6B	Baichuan2-7B-Chat
中文表达流畅度	表达自然，条理清楚	表达完整但偏冗长	表达较流畅，但偶有偏题扩写
语义歧义理解	对双关题表现较弱	长篇解释但未抓住核心双关	能意识到含义不同，但解释不够精准
多层逻辑推理	逻辑链混乱	将问题解释为悖论，未直接解决	结论直接，但解释较简略
指代关系理解	未能完成断句	判断摇摆，结论不清	给出结论但方向错误

对比维度	Qwen-7B-Chat	ChatGLM3-6B	Baichuan2-7B-Chat
一词多义理解	表现最好，解释较全面	能解释部分含义但角色分配不准	出现额外扩写，偏离原对话
运行效率	加载快，生成较快	加载快，生成极慢	加载慢，生成较快

从语言表达看，Qwen 和 ChatGLM3 都能生成比较完整的中文解释，Baichuan 的回答也通顺，但更容易偏离原题。若只看自然语言流畅度，三个模型都能达到基本可读水平；但本实验的问题并不只考察流畅度，更关注模型能否准确理解中文中的歧义和特殊句式。

在语义歧义理解方面，三个模型都没有完全答出第二题的标准双关解释。题目中的两个“谁都看不上”分别可以理解为“自己看不上别人”和“别人看不上自己”，这是中文幽默表达中的视角反转。Qwen 将其解释为“没有合适的人选”，ChatGLM3 解释为“要求过高”和“过于挑剔”，Baichuan 则解释为个人主观意愿和社会现实压力。它们都能感知两处表达可能不同，但没有准确抓住最核心的双向关系。

在多层逻辑推理方面，Baichuan 的回答最直接，指出“他不知道”；Qwen 和 ChatGLM3 都出现了较明显的逻辑混乱。Qwen 的回答中多次改变“知道”的对象，最后没有形成清楚结论。ChatGLM3 把问题解释为悖论，但题目更像嵌套认知关系解析，不一定需要上升为悖论。因此，该题反映出模型对嵌套逻辑的处理仍然容易受自然语言表面结构干扰。

在指代关系理解方面，三个模型表现都不理想。第四题需要先断句，再判断“白白喜欢他，可她就是不说”中的“她”更可能指白白。Qwen 直接表示上下文不足，ChatGLM3 的结论不稳定，Baichuan 则判断为明明喜欢白白。该题说明，当中文句子刻意去掉标点并重复使用相同汉字时，模型很容易无法正确还原句法结构。

在一词多义理解方面，Qwen 的表现最好，能够较准确地区分“意图”“敷衍表示”“不够朋友或不够诚意”“事情不大”等含义。ChatGLM3 也能解释部分含义，但对对话角色和具体语境的对应不够准确。Baichuan 则额外生成了原题没有出现的新对话，导致分析对象发生偏移。综合来看，一词多义题中 Qwen 的解释最贴近题目。

从运行效率看，Qwen 的综合体验最好，加载和生成都比较稳定。Baichuan 的加载耗时较长，但生成阶段较快，总体仍可接受。ChatGLM3 虽然加载最快，但生成阶段极慢，5 题总生成耗时超过 3800 秒，在 CPU Notebook 中使用成本最高。

8. 实验中遇到的问题与解决方法

实验过程中最先遇到的是依赖版本兼容问题。Qwen-7B-Chat 在加载时曾出现 `DisjunctiveConstraint` 无法从 `transformers` 中导入的错误。这个问题并不是模型权重下载不完整，而是 `transformers_stream_generator` 与 `transformers` 的版本不匹配。最终通过固定 `transformers==4.33.3`、`transformers_stream_generator==0.0.4` 和 `pydantic==1.10.13` 解决，并将这些依赖写入安装脚本，避免在云平台重复配置环境时再次出错。

第二个问题是 CPU 环境下模型推理容易给人“卡住”的感觉。模型加载完成后，终端会显示当前问题和 `prompt`，但普通 `model.chat()` 通常要等完整回答生成完才返回，中间不会持续打印 `token`。因此终端长时间没有新输出，并不一定表示程序停止，也可能是模型正在逐 `token` 生成答案。为了解决这个观察困难，脚本加入了 `--stream` 参数；当模型支持流式接口时，可以边生成边显示内容，更容易判断程序是在运行还是已经异常。

第三个问题是模型加载和文本生成的耗时差异很大。实验中出现过 `checkpoint shards` 很快加载到 100%，但真正生成回答仍然等待较久的情况。后来通过脚本记录模型加载耗时、每题生成耗时和总耗时，可以更清楚地区分到底是“加载慢”还是“生成慢”。同时，批量测试脚本改为单个模型只加载一次，然后连续回答五个问题，避免每个问题都重新加载模型造成额外开销。

第四个问题是精度和线程数对 CPU 推理速度的影响并不稳定。虽然低精度在 GPU 上通常能加速，但 CPU 是否支持 `bfloat16` 或 `float16` 加速取决于硬件本身；如果硬件支持不好，低精度反而可能更慢。实验中最终以 `dtype auto` 作为主要设置，并根据模型尝试不同的 `num_threads` 参数，用实际耗时而不是理论判断来选择运行配置。

最后一个是云平台磁盘空间有限。三个模型都属于 6B 到 7B 级别，模型文件较大，无法稳定地同时保存在 `/mnt/data/` 中。因此实验采用单模型下载和测试流程：先下载一个模型，完成测试并保存输出和截图后，再删除该模型目录并下载下一个模型。这样既避免了空间不足，也使每个模型的测试过程更清晰。

总体来看，这些问题并没有阻止实验完成，但它们说明在 CPU Notebook 环境中部署大语言模型时，依赖版本、推理输出方式、硬件资源和模型文件管理都会影响实验体验。通过固定依赖、加入流式输出、记录耗时以及单模型下载测试，最终实验流程变得更稳定，也更便于复现和分析。

9. 总结

本次实验完成了在 ModelScope 平台上部署并测试 Qwen-7B-Chat、ChatGLM3-6B 和 Baichuan2-7B-Chat 三个开源大语言模型的过程，并基于统一中文问题进行了横向比较。实验结果表明，三个模型都能进行基本中文问答，但在中文歧义、特殊断句、多层逻辑和指代关系问题上仍存在明显差异。

综合回答质量和运行效率看，Qwen-7B-Chat 在本次实验中整体表现较均衡。它的生成速度较快，一词多义解释较准确，但在歧义和指代题上仍会出错。Baichuan2-7B-Chat 的速度也较好，部分问题回答直接，但存在偏题和自行扩写现象。ChatGLM3-6B 的表达较完整，但 CPU 生成耗时过长，并且准确性没有明显优势。

因此，本实验的结论是：在当前 CPU Notebook 条件下，Qwen-7B-Chat 更适合作为作业展示模型；Baichuan2-7B-Chat 可以作为速度和回答风格的对照；ChatGLM3-6B 虽然能够运行，但在 CPU 环境中的生成效率较低。对于真实应用，如果希望获得更稳定和更快的大模型推理体验，仍然更适合使用 GPU 环境或更小规模的模型。